FH University of Applied Sciences TECHNIKUM WIEN

# Structured Programming Lab

## Bachelor Program Business Informatics

*Department of Computer Science*

# Formatted Output

# Printing Functions

# PrintDemo

```java
public class PrintDemo {
    public static void main(String[] args) {
        System.out.println("Hello");
        System.out.print("World!");
        System.out.println();
        System.out.printf("Goodbye World!");
    }
}
```

# OutputFunctionsDemo - Output

```
Hello
World!
Goodbye World!
```

# OutputFunctionsDemo - Summary

Table 1: Output functions

| output function | description |
|---|---|
| `print` | prints the argument (of any type) |
| `println` | prints the argument and starts a new line |
| `printf` | formatted output - use format strings |

# Special Characters

# SpecialCharacterDemo

```
1  package FormattedOutput;
2
3  public class SpecialCharacterDemo {
4
5      public static void main(String[] args) {
6          System.out.printf("Eins");
7          System.out.printf("\nZwei");
8          System.out.printf("\nDrei\nVier\nFünf");
9          System.out.printf("\n\\ \"");
10         System.out.printf("\n\"\\\\ \\\" \"");
11         System.out.printf("\n%%");
12     }
13 }
```

# SpecialCharacterDemo - Ouput

```
Eins
Zwei
Drei
Vier
Fünf
\ "
"\\ \" "
%
```

# Special Characters - Summary

Most special characters are **escaped** with a backslash (\).

Table 2: Escape sequences for special characters

| escape sequence | character description | character appearance (if printable) |
|---|---|---|
| `\\` | backslash | \ |
| `\"` | double quote | " |
| `\n` | new line | |
| `\t` | tabulator | |
| `%%` | percent sign[1] | % |

[1] in `printf`

# Formatting Integer Values

# IntegerFormattingDemo

```java
public class IntegerFormattingDemo {

    public static void main(String[] args) {
        System.out.printf("Freitag, 15.09.2017\n");
        System.out.printf("Freitag, %d.09.2017\n", 15);
        System.out.printf("Freitag, %d.%d.%d\n", 15, 9, 2017);
        System.out.printf("Freitag, %d.%2d.%d\n", 15, 9, 2017);
        System.out.printf("Freitag, %d.%02d.%d\n", 15, 9, 2017);
        System.out.printf("Freitag, %d.%-2d.%d\n", 15, 9, 2017);
        System.out.printf("Freitag, %d.%+2d.%d\n", 15, 9, 2017);
        System.out.printf("Freitag, %d.%+-8d.%d\n", 15, -9, 2017);

        System.out.printf("Freitag, %02d.%02d.%4d\n\n", 15, 9, 2017);
    }
}
```

# IntegerFormattingDemo - Output

```
Freitag, 15.09.2017
Freitag, 15.09.2017
Freitag, 15.9.2017
Freitag, 15. 9.2017
Freitag, 15.09.2017
Freitag, 15.9 .2017
Freitag, 15.+9.2017
Freitag, 15.-9      .2017
Freitag, 15.09.2017
```

# Formatting Integer - Summary I

- with the `printf()` function we can define formats with **format strings**
- declare placeholders for integer values with `%d`
- define a (minimum) **width** for a value by adding a number, e.g. `%6d` (print an integer number in decimal format with width 6
- if the value needs less characters we have several options
  - `%6d` leading spaces
  - `%-6d` left aligned
  - `%06d` leading zeros
- to print the sign use `%+d`
- (not all) combinations are possible
- we can have several placeholders in a format string
- values are mapped to placeholders by sequence of appearance

# Formatting Integer - Summary II

Table 3: Integer formatting examples

| format string | description | example output[2] (for value 123) |
|---|---|---|
| `%5d` | minimum width 5 | `|  123|` |
| `%05d` | leading zeros | `|00123|` |
| `%+5d` | print sign | `| +123|` |
| `%-5d` | left aligned | `|123  |` |
| `%x` | in hexadecimal | `|  0x7b|` |
| `%X` | in hexadecimal | `|  0x7B|` |

[2] vertical line (|) is a delimiter to illustrate alignment

# Formatting Strings

# StringFormattingDemo

```java
public class StringFormattingDemo {
    public static void main(String[] args) {

        System.out.printf("|%s|\n", "Freitag");
        System.out.printf("|%10s|\n", "Freitag");
        System.out.printf("|%-10s|\n", "Freitag");
        System.out.printf("|%S|\n", "Freitag");
        System.out.printf("|%2.3S|\n", "Freitag");

    }
}
```

# StringFormattingDemo - Output

```
|Freitag|
|   Freitag|
|Freitag   |
|FREITAG|
|FRE|
```

# StringFormattingDemo - Summary I

- declare placeholders for strings with `%s`
- define a (minimum) **width** for a string by adding a number, e.g. `%6s` (print a string with minimum width 6)
- if the string has less characters we can left align with `-`, e.g. `%-6s`
- define a maximum width for a string by adding a **precision**, e.g. `%.6s` (print a string with maximum width 6), longer strings are truncated
- combine width and precision, e.g. `%6.6s` (print a string exactly 6 characters wide)
- use `%S` to convert to upper case
- we can have several placeholders in a format string
- values are mapped to placeholders by sequence of appearance
- (not all) combinations are possible

# StringFormattingDemo - Summary II

Table 4: String formating options

| format string | description | example output[3] (for `lorem ipsum`) |
|---|---|---|
| `%15s` | minimum width 15 | `\|    lorem ipsum\|` |
| `%-15s` | left aligned | `\|lorem ipsum    \|` |
| `%10s` | minimum width 10 | `\|lorem ipsum\|` |
| `%S` | capital letters | `\|LOREM IPSUM\|` |
| `%.3s` | maximum width 3 | `\|lor\|` |
| `%5.5s` | exactly 5 wide | `\|lorem\|` |

[3] vertical line (|) is a delimiter to illustrate alignment

# Formatting Floating Point Values

# FloatingPointFormattingDemo

```java
package FormattedOutput;

public class FloatingPointFormattingDemo {

    public static void main(String[] args) {

        System.out.printf("(19.2°C)\n");
        System.out.printf("(%f°C)\n", 19.2);
        System.out.printf("(%.2f°C)\n", 19.2);
        System.out.printf("(%10.2f°C)\n", 19.2);
        System.out.printf("(%+010.2f°C)\n", 19.2);
    }
}
```

# FloatingPointFormattingDemo - Output

```
(19.2°C)
(19.200000°C)
(19.20°C)
(     19.20°C)
(+000019.20°C)
```

# FloatingPointFormattingDemo - Summary I

- declare placeholders for floating point values with `%f`
- define a (minimum) **width** for a floating point value by adding a number, e.g. `%6f` (print a floating point value with minimum width 6)
- if the value needs less characters we can left align with `-`, e.g. `%-6f`
- if the value needs less characters we can add leading zeros with `0`, e.g. `%06f`
- define a number of decimal places by adding a **precision**, e.g. `%.6f` (print a floating point value with 6 decimal places), values are rounded
- combine width and precision, e.g. `%10.6f` (print a floating point value with minimum 10 characters, 6 of which are decimal places)
- we can have several placeholders in a format string
- values are mapped to placeholders by sequence of appearance
- (not all) combinations are possible

# FloatingPointFormattingDemo - Summary II

Table 5: Floating point values formatting examples

| format string | description | example output[4] (value: 19.2) |
|---|---|---|
| `%.2f` | 2 decimal places | `\|19.20 \|` |
| `%6.2f` | minimum width 6 | `\| 19.20\|` |
| `%06.2f` | leading zeros | `\|019.20\|` |
| `%+6.2f` | print sign | `\|+19.20\|` |

[4] vertical line (|) is a delimiter to illustrate alignment

# Formatted Output - Summary

# Format Sepcifications

`printf` Syntax:

```
System.out.printf(FORMAT_STRING, ARGUMENTS)
```

- “place holders” in format string
- place holder starts with %
- first place holder for first argument
- second place holder for second argument
- n-th place holder for n-th argument
- arguments are comma-separated

Format string Syntax:

```
%[argument_index$][flags][width][.precision]conversion
```

# Conversions

Table 6: Format string conversions

| conversion | explanation |
|---|---|
| `%d` | integer values in **d**ecimal format |
| `%f` | double (and **f**loat) values |
| `%s` | **S**trings |
| `%x` | integer values in he**x**adecimal format |
| `%n` | platform-specific line separator |
| `%%` | a percent sign (%) |

# Width and Precision

- **minimum** width
- precision
  - **maximum** number of characters when used with `%s`
  - **exact** number of character after decimal separator when used with `%f`

# Flags

Table 7: Format strings flags

| Flag | Description |
|---|---|
| - | left justified |
| 0 | zero padded (numbers) |
| (space) | space padded (numbers) |
| + | always show sign (numbers) |

For more (and more detailed information) see the Java documentation.